Production Server Failure and Recovery

Complete Standard Operating Procedure, Deep Investigation Runbook & Root Cause Analysis (RCA)

PLATFORM	INCIDENT SEVERITY	SERVICE LAYER	DOCUMENT VERSION
Ubuntu 22.04 / 24.04 LTS	P1 - CRITICAL	Reverse Proxy, Web & DB	v1.0 (Production-Ready)

1. Executive Summary & Problem Scope

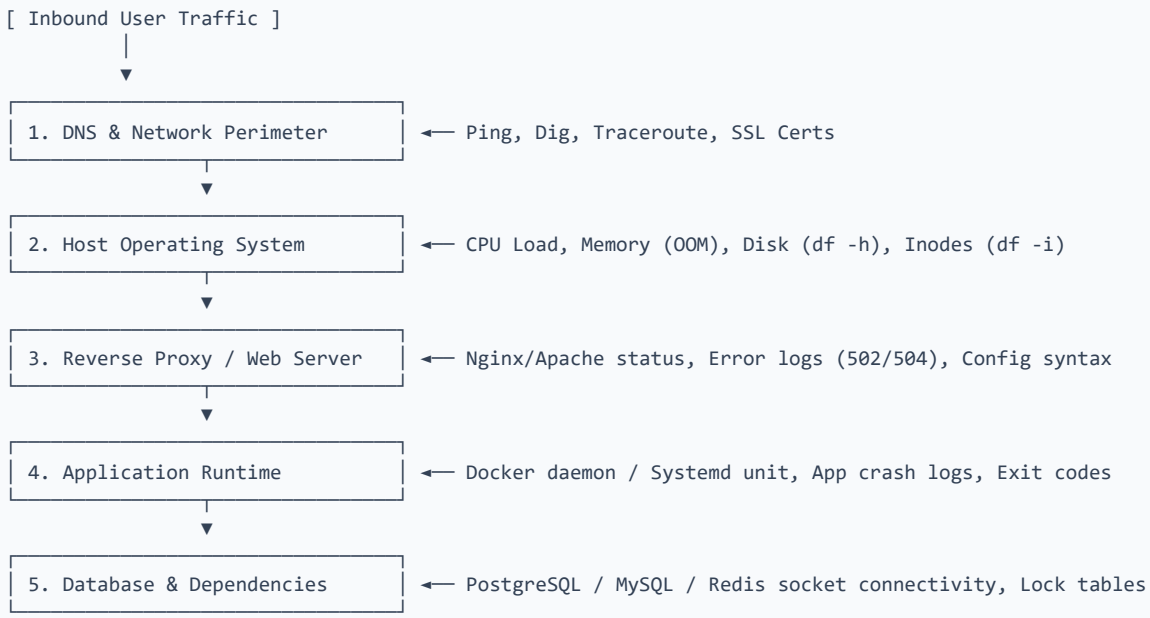
In enterprise environments, sudden application inaccessibility leads to direct business disruption and SLA penalties. When an application on an Ubuntu production host fails unexpectedly, engineers must avoid haphazard restarts or speculative changes. Instead, a deterministic, layered troubleshooting sequence must be executed across network, operating system, reverse proxy, application process, and persistence tiers.

This documentation provides the complete engineering runbook to triage the outage, isolate the root cause, execute safe recovery procedures, verify system health, and submit a formal Root Cause Analysis (RCA) report.

> **Guiding Principle:** Preserve diagnostic evidence (kernel messages, systemd journal, container logs) prior to performing service reboots or cache flushes.

2. Incident Architecture & Diagnostics Flow

Inbound client requests traverse distinct architectural layers. When an outage occurs, troubleshooting should methodically progress through these layers from ingress to data persistence:



3. Phase 1: Immediate Triage & Incident Communication

1. **Declare P1 Severity:** Notify the on-call channel (#incident-prod) and internal stakeholders with an initial notification:

   > *"Investigating reports of users unable to access production web service. Server connection and diagnostics underway. Updates every 15 minutes."*

2. **Ingress Diagnostic Probe:**

```
# Test remote HTTP response, TLS handshake, and response headers
curl -Iv https://app.example.com/health

# If custom IP or port:
curl -Iv http://<SERVER_IP>:8080/
```

3. **Interpret Initial HTTP Status Signals:**
   - `Could not resolve host`: DNS failure or domain registration issue.
   - `Connection refused / Operation timed out`: Server offline, port not bound, or security group/firewall blocking packets.
   - `HTTP 502 Bad Gateway`: Reverse proxy (Nginx) is alive, but upstream backend application is offline or refusing connections.
   - `HTTP 504 Gateway Timeout`: Backend process is locked, deadlocked, or timing out.
   - `HTTP 500 Internal Server Error`: Backend is reachable, but failing unhandled runtime code exceptions.

4. Phase 2: Systematic Layer-by-Layer Investigation

Step 2.1: Establish Remote Host Connection

```
# Secure Shell into target Ubuntu host
ssh -i ~/.ssh/production_key.pem ubuntu@<SERVER_IP>
```

Step 2.2: Host OS Resource & Kernel Diagnostics

```
# 1. Check uptime, load average, and process states
uptime
top -b -n 1 | head -n 25

# 2. Check memory utilization and swap
free -h

# 3. Check for Out-Of-Memory (OOM) Killer execution
sudo dmesg -T | grep -i -E "oom|out of memory|killed process"
sudo journalctl -k --since "2 hours ago" | grep -i "killed process"

# 4. Check disk block usage and inode allocation
df -h
df -i
```

> **High-Risk Diagnostic:** If `df -h` indicates `100%` usage on root partition (`/`), system logging, session creation, and lockfiles halt immediately. If `dmesg` reports `Killed process <pid> (node/python)` , the application was killed by the OS due to RAM exhaustion.

Step 2.3: Network Sockets & Port Bindings

```
# Inspect listening ports and process ownership
sudo ss -tulpn | grep -E ':(80|443|3000|5000|8080)'

# Check local firewall status
sudo ufw status verbose
```

Step 2.4: Reverse Proxy & Web Server Diagnostics

```
# Check Nginx status and validate configuration files
sudo systemctl status nginx
sudo nginx -t

# Inspect recent Nginx error log events
sudo tail -n 100 /var/log/nginx/error.log
```

Step 2.5: Application Runtime Diagnostics

Depending on deployment architecture:

```
# If managed by systemd:
sudo systemctl status <app-service-name>
sudo journalctl -u <app-service-name> -n 150 --no-pager

# If running inside Docker:
docker ps -a
docker inspect <container-name> --format 'Status: {{.State.Status}} | ExitCode: {{.State.ExitCode}} | Error: {{
docker logs --tail 100 --timestamps <container-name>
```

5. Phase 3: Root Cause Decision Matrix

Observed Symptom	Diagnostic Indicators	Determined Root Cause	Resolution Path
HTTP 502 Bad Gateway	`dmesg -T`: Out of memory kill. `docker ps -a`: Exit code 137.	**OOM Killer Invocations:** Unbounded memory consumption or memory leak triggered kernel kill.	Provision swap file, adjust memory quotas, restart backend service.
Complete System Hang / Freeze	`df -h`: `/dev/root` at 100%. Logs state: `No space left on device`.	**Disk Saturation:** Massive unrotated log files or orphaned docker build layers filled disk.	Vacuum journalctl, prune docker caches, truncate bloated log files.
HTTP 500 Internal Server Error	App log: `ConnectionRefusedError: [Errno 111]` to 5432 / 3306.	**Database Failure:** PostgreSQL / MySQL daemon crashed or exhausted connection pool.	Restart DB daemon, clear zombie connections, optimize pool size.
SSL Handshake Failure	`curl: (60) SSL certificate problem: certificate has expired`.	**Expired TLS Certificate:** Automated Let's Encrypt / Certbot cron failed.	Force certificate renewal via Certbot and reload Nginx.

6. Phase 4: Safe Remediation & Recovery Procedures

Procedure A: Resolving Disk Space Saturation (100% Full)

```
# 1. Safely vacuum systemd journal logs older than 2 days
sudo journalctl --vacuum-time=2d

# 2. Prune unused Docker containers, images, and builder caches
docker system prune -af --volumes

# 3. Truncate oversized logs without removing active file descriptors
sudo find /var/log -type f -name "*.log" -size +500M -exec truncate -s 0 {} +

# 4. Verify disk usage is below 80%
df -h /
```

Procedure B: Resolving Memory Exhaustion & OOM Crashes

```
# 1. Create and mount an emergency 4GB swap space if none exists
if [ $(swapon --show | wc -l) -eq 0 ]; then
  sudo fallocate -l 4G /swapfile
  sudo chmod 600 /swapfile
  sudo mkswap /swapfile
  sudo swapon /swapfile
  echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
fi

# 2. Restart backend application process
sudo systemctl restart <app-service-name>
# Or Docker:
docker restart <container-name>

# 3. Restart Reverse Proxy
sudo systemctl restart nginx
```

Procedure C: Rolling Back Bad Deployment

```
# Check recent git commits or container tags
git log -n 5 --oneline
# Rollback to last known healthy commit/tag
git checkout <stable-commit-hash>
sudo systemctl restart <app-service-name>
```

7. Phase 5: Health Verification & Service Validation

1. **Internal Synthetic Health Check:**

```
# Verify backend responds locally on loopback
curl -I http://127.0.0.1:5000/health
```

2. **External Public Health Check:**

```
# Verify public reverse proxy ingress
curl -Iv https://app.example.com/health
```

   Expected output: `HTTP/2 200 OK`

3. **Log Stream Validation:**

```
# Monitor live access logs for recurring HTTP 4xx or 5xx errors
sudo tail -f /var/log/nginx/access.log | grep -E '" [45][0-9]{2} '
```

4. **Stakeholder Broadcast:** Post resolution notice with verified metric dashboards to internal channels.

8. Phase 6: Production Root Cause Analysis (RCA) Report

INCIDENT POST-MORTEM & ROOT CAUSE ANALYSIS	
Incident Identifier	INC-20260928-01
Affected Application	Production Web Application & Customer API
Severity Level	P1 - Critical (Complete Outage)
Outage Window	2026-09-28 18:00 IST to 18:40 IST (40 Minutes)
Incident Commander	DevOps / SRE Engineer
Resolution Status	Resolved (Full Service Restored)

8.1 Executive Summary

On 2026-09-28 at 18:00 IST, users were unable to access the production application, encountering HTTP 502 Bad Gateway error responses. Triage revealed that the primary backend application process was terminated by the Linux kernel Out-Of-Memory (OOM) killer due to physical memory exhaustion provoked by an unpaginated background reporting query. Services were restored by provisioning emergency swap space, recycling the backend and Nginx daemons, and throttling batch queries. Full system recovery was confirmed at 18:40 IST.

8.2 Chronological Timeline

- **18:00:00 IST:** Automated alerts fire in Slack; error rate spikes to 100%.
- **18:05:00 IST:** Incident acknowledged; on-call begins external verification.
- **18:08:00 IST:** `curl` probe confirms `HTTP 502 Bad Gateway`.
- **18:12:00 IST:** SSH session established. `uptime` normal, but `free -h` indicates 99% RAM usage with 0B swap.
- **18:16:00 IST:** `dmesg -T` confirms kernel OOM kill on PID 14209 (`backend-api`).

- **18:24:00 IST:** Root cause traced to unpaginated query loading 450,000 transaction records into RAM.
- **18:28:00 IST:** 4GB swap space configured and active.
- **18:32:00 IST:** Backend process restarted; TCP port bound successfully.
- **18:35:00 IST:** Nginx reloaded; health check returns `HTTP 200 OK`.
- **18:40:00 IST:** Public ingress verified across all geographic zones; incident resolved.

8.3 5-Whys Root Cause Analysis

1. **Why did users experience an outage?** Reverse proxy returned 502 Bad Gateway because the backend service was offline.
2. **Why was the backend service offline?** The Linux kernel abruptly terminated the process with SIGKILL.
3. **Why did the kernel terminate the process?** Physical RAM was completely saturated, invoking the OOM Killer.
4. **Why was RAM saturated?** An ad-hoc export job loaded 450,000 database rows into application heap memory without pagination.
5. **Why was the job allowed to exhaust host memory?** The application endpoint lacked chunked streaming, and the host lacked swap space and systemd cgroup memory limits.

8.4 Corrective & Preventative Actions (CAPA)

Action Item	Classification	Owner	Target Date
Configure permanent 4GB swap on all production Ubuntu hosts	Preventative	DevOps	2026-09-29
Configure systemd unit memory quotas (`MemoryMax=6G` , `Restart=always`)	Preventative	DevOps	2026-09-29
Refactor transaction export endpoint to stream data with pagination	Corrective	Backend Eng	2026-09-30
Configure Prometheus alert for Host Memory > 85% for > 2 mins	Detective	SRE Team	2026-10-01
Deploy Nginx rate-limiting on batch export endpoints	Preventative	DevOps	2026-10-02

> **Sign-Off & Approval:** Incident closed and verified by Incident Commander and Engineering Team Lead.